

100 DAYS OF CODE – 4

(C-PROGRAMMING)

❖ Problem Statement:

Correct the given C program, algorithm. and flowcharts to perform multiplications of two integers without using the multiplication (*) operator. Implement the solution using repeated addition and ensure that it correctly handles positive numbers, negative numbers, and zero.

➤ Objective:

To implement multiplication of two integers using repeated addition instead of the multiplication(*) operator and verify the program using different test cases.

➤ Given Algorithm:

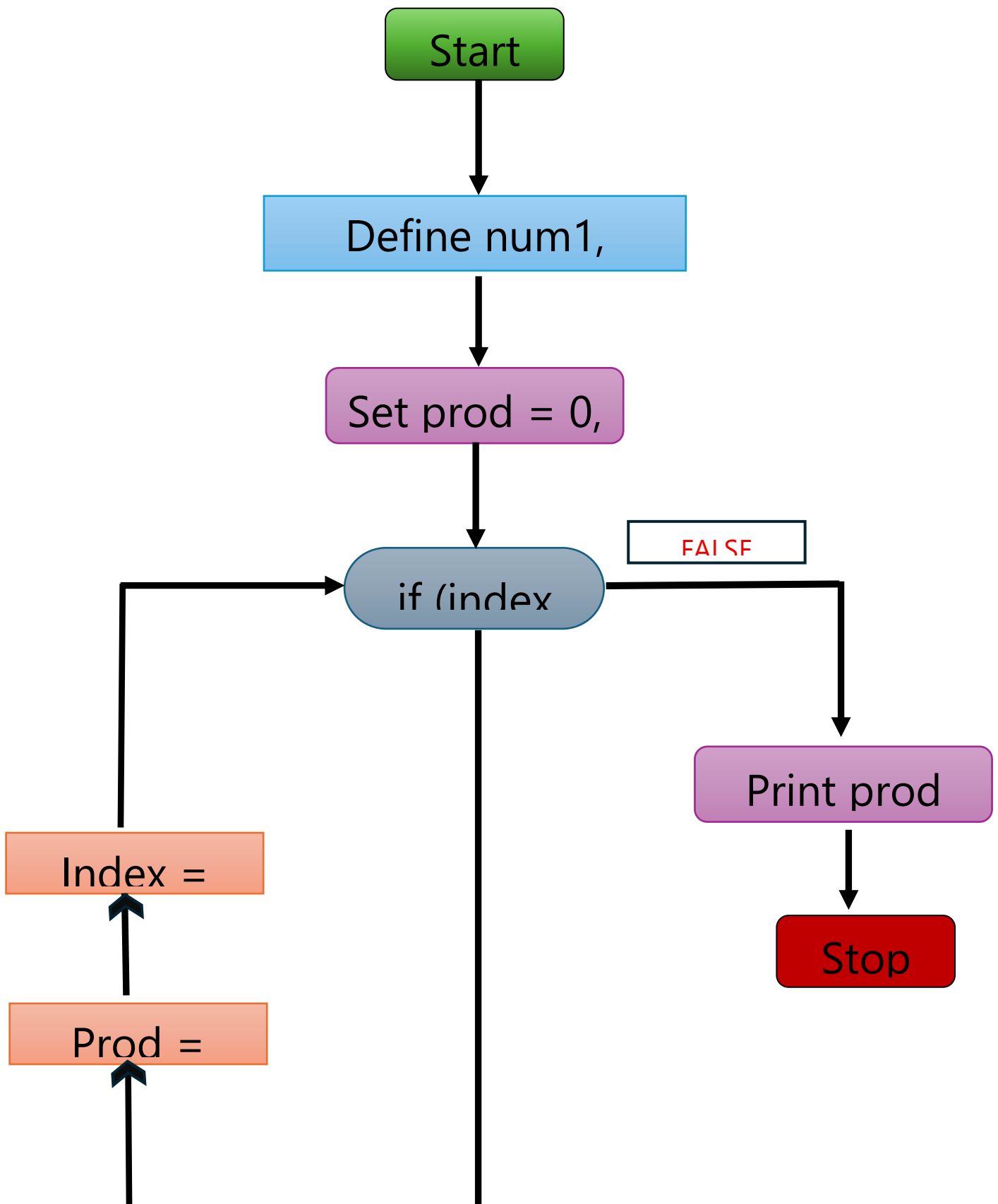
- i. Define num1, num2, prod, index as integers.
- ii. Get num1, num2.
- iii. Set prod = 0, index = 1
- iv. Repeat steps 4.1 and 4.2 until (index <= num2)
 - 4.1 prod = prod + num1
 - 4.2 index = index + 1
- v. Print prod
- vi. Stop

✓ Here are the mistakes in the given algorithm:

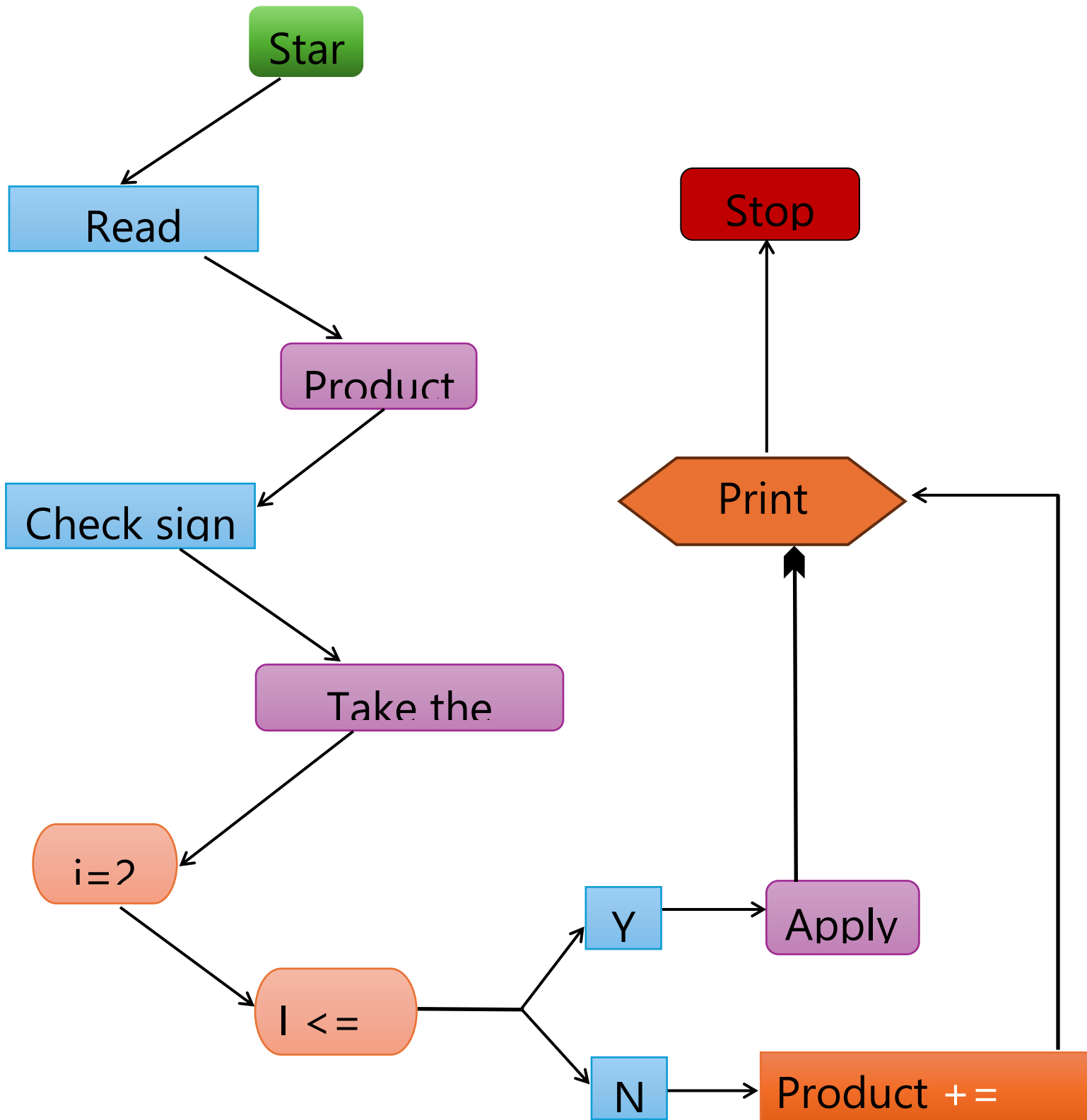
- i. It works only for positive values of num2
- ii. It does not work for negative numbers

- iii. It does not correctly handle multiplication if one of the inputs is zero
- iv. It does not determine the sign of the result

❖ Flowchart



❖ Here is the correct Flowchart:



✓ Here is the correct algorithm:

- i. Define num1, num2, prod, index, sign, a, b, small, large as integers.**
- ii. Get num1, num2.**
- iii. If (num1 = 0 OR num2 = 0) then:**
 - 3.1 Set prod = 0**
 - 3.2 Go to Step 9**
- iv. If ((num1 > 0 AND num2 > 0) OR (num1 < 0 AND num2 < 0)) then:**
 - 4.1 Set sign = 1**
 - 4.2 Else Set sign = -1**
- v. Set a = |num1|, b = |num2|.**
- vi. Set small = min(a, b), large = max(a, b).**
- vii. Set prod = 0, index = 1.**
- viii. Repeat steps 8.1 and 8.2 while (index <= small):**

8.1 prod = prod + large

8.2 index = index + 1

ix. Set prod = prod * sign.

x. Print prod.

xi. Stop.

❖ Corrected C Program:

```
Session Actions Edit View Help
GNU nano 9.0
#include <stdio.h>

int main() {
    int num1, num2;
    int product = 0;
    int i;
    int temp;

    printf("Enter any two integers: ");
    scanf("%d %d", &num1, &num2);

    // If num2 is negative, flip signs of both numbers
    if (num2 < 0) {
        num1 = -num1;
        num2 = -num2;
    }

    // Swap if num1 is smaller than num2 (to ensure minimum additions)
    // We check positive values manually using simple if-else
    int pos_num1 = num1;
    int pos_num2 = num2;

    if (pos_num1 < 0) {
        pos_num1 = -pos_num1;
    }
    if (pos_num2 < 0) {
        pos_num2 = -pos_num2;
    }

    if (pos_num1 < pos_num2) {
        temp = num1;
        num1 = num2;
        num2 = temp;
    }

    // Standard for loop
    for (i = 1; i <= num2; i++) {
        product = product + num1;
    }

    printf("Product = %d\n", product);
    return 0;
}
```

- Test case 1:

```
(kali@kali)-[~]
$ cc hdoc4.c

(kali@kali)-[~]
$ ./a.out
Enter any two integers: 12 3
Product = 36
```

- Test case 2

```
(kali@kali)-[~]
$ ./a.out
Enter any two integers: 3 12
Product = 36
```

- Test case 4

```
(kali@kali)-[~]  
└─$ ./a.out  
Enter any two integers: 6 -5  
Product = -30
```

- Test case 5

```
(kali@kali)-[~]  
└─$ ./a.out  
Enter any two integers: -9 -4  
Product = 36
```

- Test case 5

```
(kali@kali)-[~]  
└─$ ./a.out  
Enter any two integers: 8 0  
Product = 0
```

\\These were few test cases.

➤ Conclusion

The original algorithm and flowchart only worked for two positive integers and never minimised the

number of additions. The corrected version adds early zero check, a sign-determination step, and uses the smaller magnitude as the loop counter (with the larger magnitude as the addend). This guarantees correct results for all sign combinations and zero, using the minimum possible number of additions, verified against 5 test cases above.